TSAR Project Assignment Part 2 - Group 3

Arpad Broglie Teodor Glisic

2026

Table of contents

Introduction

In this project, we explore the pre-processing step of data wrangling. We do this using dplyr and tidyr. Real-world datasets often are filled with missing values and anomalies. When performing calculations on these dataset this may result in problems. Therefore we first have to perform automated Exploratory Data Analysis (EDA) using the DataExplorer package. This will help to establish an overview of the missing data. Subsequently, we manually wrangle the data to identify and quantify the "dirty" values. We consider empty strings, spaces, NaN values and statistical outliers across all categorical and numerical attributes as dirty values.

The goal is to assess the dataset's quality to inform future data cleaning decisions.

Package installation

```r
# Check for missing packages, install them if necessary,
# and load them into the environment.
if (!require(dplyr)) install.packages("dplyr")
if (!require(tidyr)) install.packages("tidyr")
if (!require(data.table)) install.packages("data.table")
if (!require(DataExplorer)) install.packages("DataExplorer")
if (!require(ggplot2)) install.packages("ggplot2")

library(dplyr)
library(ggplot2)
library(tidyr)
library(data.table)
library(DataExplorer)
```

Task 1: Exploratory Data Analysis

We begin by loading our private dataset, `myLCdata`, and applying a custom `dirtify` function using our group ID to introduce the anomalies. We then generate an automated EDA report using `DataExplorer::create_report()`. Figure 1 displays the "Missing Data Profile" generated by this report. This profile primarily highlights standard `NA` values, revealing that the `home_ownership` feature is the most complete and categorized as "good". However, this automated tool does not capture the full extent of the hidden special strings, necessitating our manual wrangling step.

```r
# Load the private dataset
myLCdata <- fread("myLCdata.csv")

# Apply the dirtyfy function using our group ID (3)
dirtify <- readRDS(file = "dirtyfy.rds")
myLCdata_dirty <- dirtify(myLCdata, 3)

# Executing the DataExplorer::create_report()
# on the dirtified myLCdata in the console
# create_report(myLCdata_dirty)
```

Result from DataExplorer::create_report()

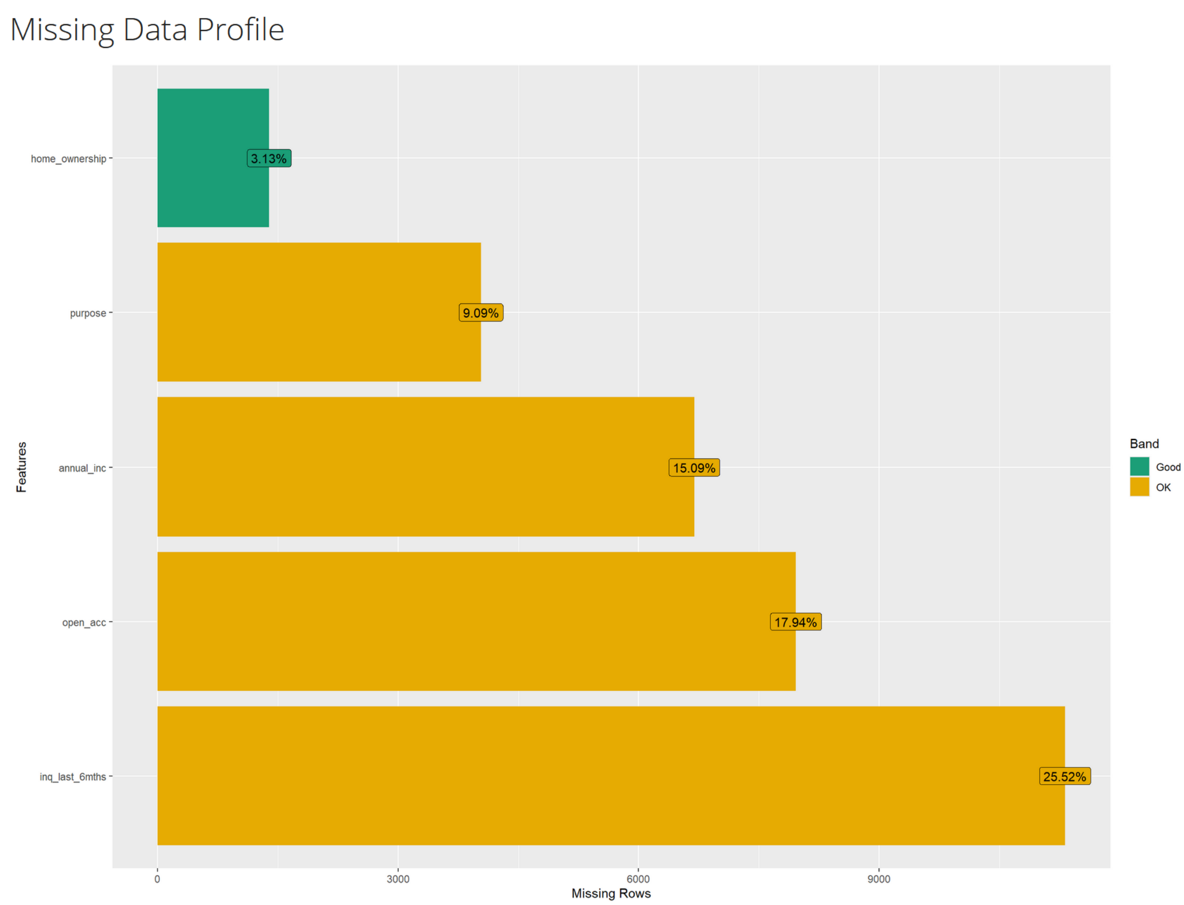

Figure 1: Presents an overview of missing rows by features of the myLCdata_dirty variable. The home_ownership feature is the most complete and the only one categorized as good.

Task 2: Data Wrangling and Visualization

```r
glimpse(myLCdata_dirty)
```

```
Rows: 44,369
Columns: 5
$ annual_inc      <dbl> 60000, 18576, NaN, 35000, 125000, 144000, 205000, 11800~
$ open_acc        <dbl> 6, NaN, 15, 3, NaN, 8, 9, 18, NaN, 3, 11, 10, 15, 5, Na~
```

```
$ purpose        <chr> "credit_card", "", "small_business", "home_improvement"~
$ home_ownership <chr> "OWN", "RENT", "RENT", "OWN", "RENT", " ", "MORTGAGE", ~
$ inq_last_6mths <dbl> 0, NA, 2, 1, NA, NA, 0, 0, NA, 0, 0, NA, NA, 0, 0, 0, 1~
```

```r
str(myLCdata_dirty)
```

```
Classes 'data.table' and 'data.frame':  44369 obs. of  5 variables:
 $ annual_inc    : num  60000 18576 NaN 35000 125000 ...
 $ open_acc      : num  6 NaN 15 3 NaN 8 9 18 NaN 3 ...
 $ purpose       : chr  "credit_card" "" "small_business" "home_improvement" ...
 $ home_ownership: chr  "OWN" "RENT" "RENT" "OWN" ...
 $ inq_last_6mths: num  0 NA 2 1 NA NaN 0 0 NaN 0 ...
 - attr(*, ".internal.selfref")=<externalptr>
```

```r
summary(myLCdata_dirty)
```

```
   annual_inc        open_acc       purpose          home_ownership    
 Min.   :   4000   Min.   : 1.00   Length:44369       Length:44369      
 1st Qu.:  45931   1st Qu.: 8.00   Class :character   Class :character  
 Median :  65000   Median :11.00   Mode  :character   Mode  :character  
 Mean   :  75314   Mean   :11.58                                        
 3rd Qu.:  90000   3rd Qu.:14.00                                        
 Max.   :6000000   Max.   :76.00                                        
 NA's   :6694      NA's   :7962                                         
 inq_last_6mths  
 Min.   : 0.0000  
 1st Qu.: 0.0000  
 Median : 0.0000  
 Mean   : 0.6951  
 3rd Qu.: 1.0000  
 Max.   :31.0000  
 NA's   :11321    
```

Table 1: Explanation of the variables (categorical or numeric)

Variable	Explanation
annual_inc	numerical
open_acc	numerical
purpose	categorical
home_ownership	categorical
inq_last_6mths	numerical

Empty categorical features can contain the following values:

- **`NA` or Not Available:** Indicates missing values that R recognizes as missing.
- **The string `"n/a"`:** We pretend this string was used in the source data set to indicate missing data. Since it is a string R does not recognize it as missing.
- **A space `" "`:** Another special string.
- **The empty string `""`:** Another special string.

Empty numerical features can contain the following values:

- **`NA`:** Standard missing values.
- **`NaN` or Not a Number:** Results from invalid computations such as 0/0.
- **Outliers:** These were actually not ingested but already present in the data. There is no rigid mathematical definition of what constitutes an outlier. In this task we use Tukey's fences. We count values as outliers if they lie outside the whiskers of the boxplot.

Overall NA per feature

```
myLCdata_dirty %>% summarise(across(everything(), ~sum(is.na(.))))
```

```
  annual_inc open_acc purpose home_ownership inq_last_6mths
1       6694     7962    4033           1390          11321
```

Categorical features (purpose, home_ownership)

For our categorical variables (`purpose` and `home_ownership`), we calculate the relative frequencies of various missing data representations. As seen in Figure 2, the data contains standard `NA`s, which R natively recognizes as missing. During a future cleaning phase, we would need to decide whether to impute these values or remove the affected rows or columns. We also identified hidden missing data disguised as `"n/a"` strings, empty strings, and spaces. Notably, spaces may or may not truly indicate missing data, and text strings like "n/a" must be explicitly transformed into NA for R to process them correctly.

```
# Calculate percentages for all categorical columns dynamically
myLCdata_dirty_values_categorical <- myLCdata_dirty %>%
  select(purpose, home_ownership) %>%
  pivot_longer(cols = everything(),
               names_to = "columns",
               values_to = "empty_value") %>%
  filter(if_any(everything(), ~ . %in% c(" ", "", "n/a") | is.na(.))) %>%
  mutate(empty_value = case_when(
    empty_value == "" ~ "empty_string",
    empty_value == " " ~ "single_space",
    is.na(empty_value) ~ "NA_value",
    empty_value == "n/a" ~ "n/a"
  )) %>%
  group_by(columns, empty_value) %>% summarise(dirty_value_count = n()) %>%
  pivot_wider(names_from = empty_value,
              values_from=dirty_value_count,
              names_prefix = "percentage_") %>%
  mutate(across(where(is.numeric), ~. / nrow(myLCdata_dirty)))

head(myLCdata_dirty_values_categorical)
```

```
# A tibble: 2 x 5
# Groups:   columns [2]
  columns        percentage_NA_value percentage_empty_string `percentage_n/a`
  <chr>                        <dbl>                   <dbl>            <dbl>
1 home_ownership              0.0313                  0.0935           0.0176
2 purpose                     0.0909                  0.0520           0.0903
# i 1 more variable: percentage_single_space <dbl>
```

```
# Plotting the categorical data
ggplot_data_categorical <- myLCdata_dirty_values_categorical %>%
  pivot_longer(cols=-columns,
               names_to = "empty_value",
               values_to = "proportion")

ggplot(ggplot_data_categorical) +
  geom_col(aes(x = columns, y = proportion)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
```

```
  facet_wrap(~empty_value, ncol = 1) +
  labs(y = "Proportional distribution", x = "Categorical attribute")
```

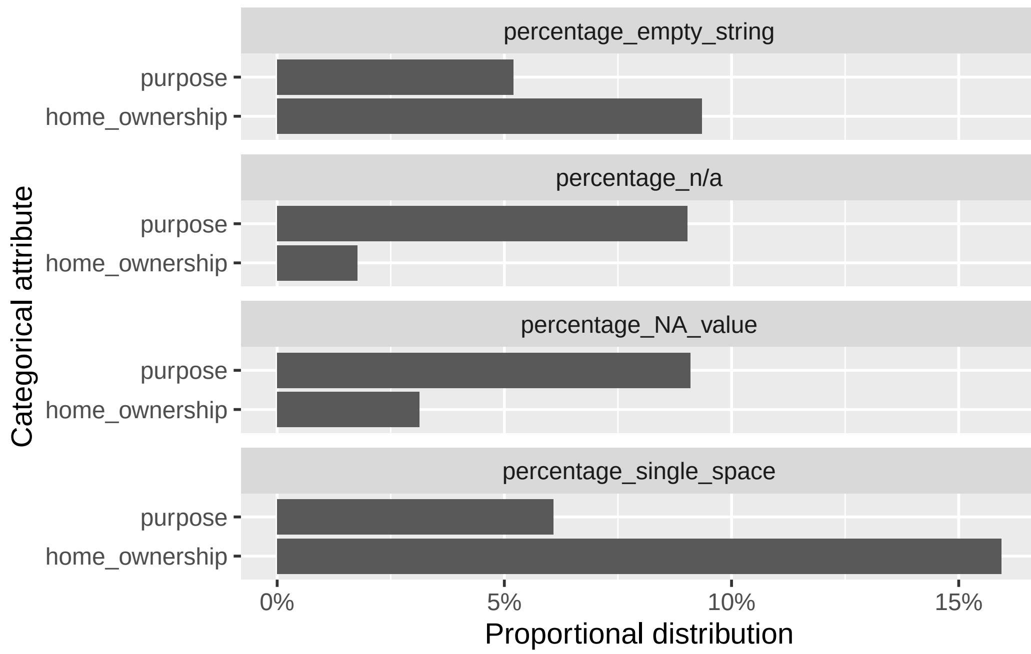

Figure 2: Percentage of special values in the categorical attributes of myLCdata

Numerical Features

We also evaluate our numerical attributes (`annual_inc`, `open_acc`, `inq_last_6mths`) for anomalies. Figure 3 details the proportion of standard `NA`s, `NaN`s resulting from invalid computations, and statistical outliers. Outliers were identified using Tukey's fences, meaning any value falling outside the whiskers of a standard boxplot was counted. While these outliers will need to be inspected closely during data cleaning to determine if they are errors or valuable edge cases (such as a medical anomaly in healthcare data), `NaN`s will likely need to be corrected or treated as standard missing values.

```
# Calculate percentages for all numerical columns dynamically
analysis_missing_numeric <- myLCdata_dirty %>%
  summarise(
    across(
```

```r
      where(is.numeric),
      list(
        perc_NA = ~ sum(is.na(.) & !is.nan(.)) / n(),
        perc_NaN = ~ sum(is.nan(.)) / n(),
        perc_outliers = ~ length(boxplot.stats(.)$out) / n()
      )
    )
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("attribute", ".value"),
    names_pattern = "(.*)_(perc_.*)"
  )

head(analysis_missing_numeric)
```

```
# A tibble: 3 x 4
  attribute      perc_NA perc_NaN perc_outliers
  <chr>            <dbl>    <dbl>         <dbl>
1 annual_inc      0.0494    0.101        0.0390
2 open_acc        0.0752    0.104        0.0258
3 inq_last_6mths  0.0920    0.163        0.0456
```

```r
# Plotting the numerical data
ggplot_data_numeric <- analysis_missing_numeric %>%
  pivot_longer(cols = -attribute, names_to = "type", values_to = "proportion")

ggplot(ggplot_data_numeric) +
  geom_col(aes(x = attribute, y = proportion)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  facet_wrap(~type, ncol = 1) +
  labs(y = "Proportional distribution", x = "Numerical attribute")
```

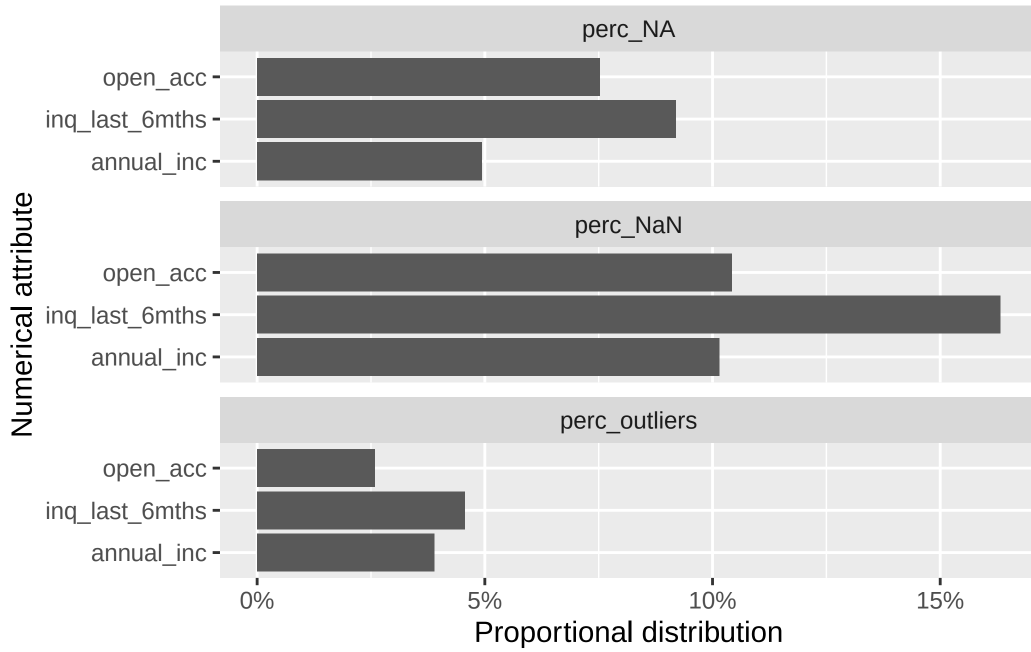

Figure 3: Percentage of special values in the numerical attributes of myLCdata

Conclusion

Our manual data wrangling successfully revealed the true extent of inconsistencies within the `myLCdata_dirty` dataset. While the automated `DataExplorer` report gave us a baseline understanding of standard missing values, our `dplyr` and `tidyr` operations uncovered a significant amount of hidden "dirty" data, such as `"n/a"` text strings and empty spaces. Furthermore, applying Tukey's fences allowed us to systematically isolate outliers across all numeric variables. This comprehensive overview of the dataset's flaws provides the necessary groundwork for the subsequent formal data cleaning phase, where we will decide how to impute, correct, or remove these anomalies to ensure accurate future analyses.